



UNIVERSIDAD DE DISEÑO, INNOVACIÓN Y TECNOLOGÍA

**Grado en Diseño y Desarrollo
de Videojuegos
(Curso 2025 - 2026)**

**Entrega: Multiplayer
(29/10/2025)**

Ramón Zabara Morales



UNIVERSIDAD DE DISEÑO,
INNOVACIÓN Y TECNOLOGÍA

Ramón Zabara Morales

Índice

Portada	1
Índice	2
Descripción general	3
Requisitos de la entrega	3
Descripción técnica	4
Diario de desarrollo	8
Anexos	8
Bibliografía	9

Descripción general

El objetivo detrás de la realización de esta práctica es el de aprender sobre el sistema multijugador de Unreal Engine 5, así como mejorar mis habilidades con dicho programa ordenando mejor el código y familiarizandome más con su sistema de eventos y las distintas opciones en las que se pueden ejecutar dichos eventos.

Requisitos de la entrega

1. Los jugadores harán daño, al menos, de dos tipos de escuelas diferentes (p. ej. Físico, frío, fuego...). El tipo de daño se puede seleccionar mediante la UI o una tecla de acceso directo, o la rueda del ratón.
2. Los jugadores van desarrollando inmunidad al daño de una escuela a medida que lo reciben (rendimiento decreciente, 100%, 50%, 25%) con una ventana de 15 segundos, que se reinicia por cada disparo. La inmunidad es total después de recibir daño del mismo tipo tres veces consecutivas, dentro de la ventana de tiempo. (Este apartado es opcional, para subir nota)
3. Los jugadores contarán con un sistema de recursos (energía, balas, cargadores). Al comenzar el juego tienen un cargador de 10 balas y 0 puntos de energía (mínimo). (Los cargadores son opcionales, para subir nota)
4. Los jugadores pueden disparar haciendo clic izdo. (un tipo de daño puntual que genera 10 puntos energía y posee 1.5 segundos de reutilización) y clic derecho (tipo de daño en área que no tiene tiempo de reutilización pero que consume 70 puntos de energía y destruye las coberturas)
5. Los jugadores poseen balas limitadas para el disparo de clic izdo. Si se gastan las balas del cargador activo, el jugador será informado y deberá recargar pulsando una tecla, siempre que tenga suficientes cargadores de repuesto. Cada recarga es de 10 balas. Si el jugador acierta dos disparos consecutivos contra el oponente, gana un cargador. Se pueden tener un máximo de cinco cargadores (1 activo + 4 reserva) (Este apartado es opcional, para subir nota)
6. Los jugadores deben poseer un sistema de salud. Si se pierde toda, el jugador muere. Durante la muerte, el jugador será informado (Se muestra el mensaje Has muerto, y se tinte la imagen de rojo y aparece una cuenta atrás de 10 segundos) durante los cuales, el jugador no podrá moverse hasta que no llegue a cero.

7. Los jugadores tendrán una interfaz de usuario gráfica donde se muestre su salud, su energía, número de balas, número de cargadores, y tiempo de reutilización.
8. Los jugadores cuya salud cae por debajo del 30% pueden usar el botón central del ratón para usar una poción que sana 70 puntos de salud, pero posee un tiempo de reutilización de 60 segundos).
9. El juego dispondrá de un combat log, donde se imprimirá por pantalla quien recibe el daño, cuánto daño recibe, el tipo de daño, la causa del daño y el causante del daño.
10. Nota: El sistema de cargadores es opcional, sólo se implementa si se desea subir nota. Si no se implementa, las balas se regenerarán automáticamente a razón de dos balas cada 5 segundos.

Descripción técnica

Para la realización de este proyecto se han necesitado de varios assets y blueprints los cuales serán comentado a continuación:

Entre los blueprints está el del first person character en el begin play se asignan los controles y se llama a un evento que se ejecuta en el servidor el cual establece los valores iniciales para el jugador y luego hay un evento que se ejecuta en el cliente que crea la UI del jugador y se la muestra. Luego están las acciones de salta, del movimiento del personaje, del ratón que tiene un evento ejecutado en el servidor para replicar la rotación en pitch con el maniquí completo para que el resto de jugadores lo vean y que dentro se llama a otra función que también se ejecuta en el servidor y que se encarga de replicar el pitch como tal, los cuales se ejecutan sólo cuando el jugador puede moverse, en este proyecto esa solo ocurre mientras el personaje esté vivo, pero en otros proyectos podría usarse para que el jugador no se mueva durante cinemáticas. Aparte de estas funciones se encuentra la función de cambiar el tipo de daño, el cual llama a un evento que se ejecuta en el servidor y lo que hace es que cuando la rueda del ratón se mueve hacia arriba se reduce el índice del tipo de daño a infligir, y cuando la rueda va hacia abajo el índice sube, teniendo el 0 y el 2 como topes, cumpliendo así el primer requisito de la entrega. Luego están las funciones de disparo, que funcionan casi igual, empezando con 10 balas, los 0 cargadores extra y 0 de energía cumpliendo así el punto 3, en el disparo primario comprueba que pueda disparar tanto por que el jugador tiene munición suficiente así como el tiempo entre disparos ya a acabado, una vez esto se ha comprobado se realiza la animación de disparar y se llama a una función que se ejecuta en el servidor la cual es la que dispara la bala y establece de nuevo el timer para poder

volver a disparar, la bala contiene un ID cuyo uso se explicará más adelante, y un tipo de daño que es el que infringe, además de un instigador y un propietario de dicha bala que es el jugador, después se resta uno a la munición y se aumenta en uno el ID actual de bala para asignárselo a la siguiente que sea disparada, tras esto se comprueba si el jugador tiene 0 balas y cargadores, y en caso positivo comienza un timer de 5 segundos para darle dos balas cumpliendo así el requisito número 10, tras esta comprobación se inicia el timer de poder volver a disparar que dura 1'5 segundos y finalmente se llama a un evento multicast que hace sonar el disparo, en cuanto a los timers estos se ejecutan también en el servidor; en cuanto a proyectil secundario esta la comprobación que hace es tener 70 o más de energía para poder dispararse, de ahí reproduce la animación de disparar del personaje y se llama a una función que se ejecuta en el servidor que dispara el disparo secundario que genera el proyectil secundario que pide los mismos parámetros que el proyectil primario, después le resta esos 70 de energía necesarios para disparar el secundario y se le suma uno al current ID, de ahí como en el proyectil primario llama a una función multicast que hace sonar el disparo, cumpliendo así la mayoría del cuarto requisito, el resto del punto se encuentra más adelante, cuando se expliquen los blueprints de los proyectiles. Pasando a la función que se encarga de recibir el daño hay dos nodos que desencadenan esta acción, el evento PointDamage y el RadialDamage los cuales llaman a una función update damage que se ejecuta en el servidor y que recibe como parámetros la clase del daño que ha recibido, el daño original que hace la bala y el damage causer, dicha función primero comprueba si el anterior disparo que recibió este player es del mismo tipo que el anterior, y en caso afirmativo aplica la inmunidad correspondiente al daño recibido y resetea el timer, en caso de que fuese un daño distinto al anterior entonces recibe el daño completo y comienza el timer para que ese tipo de daño vuelva a tener un efecto del 100% (apunte: si se dispara con un tipo, luego con otro y otra vez con el primero el daño es del 100%, dado que lo que se pretende es que se usen distintos tipos de daño, no ir negándolos todos durante un tiempo, así se consigue un gameplay activo y variado a la vez), cumpliendo así el segundo requisito de la práctica. De ahí pasamos a una función que aplica el daño y establece el tipo de daño que se ha recibido para futuras comprobaciones, y dicha función al cambiar la vida al ser esta una variable renotify cambia la UI al llamar a las funciones pertinentes. Tras esto se llama en un append de strings al nombre del objeto que recibió el daño, al daño (con la inmunidad ya aplicada) que ha recibido, al objeto que lo dañó con el damage causer, y al jugador que disparó dicha bala mediante un get owner al damage causer y todo esto se muestra en un string en consola, obteniendo así el combat log del noveno requisito. En caso de que dicho daño matase al jugador entonces se llama a un evento multicast para mostrar la animación de muerte, se le impide moverse al jugador y se setea un timer a una función ejecutada en el servidor que espera 10 segundos para revivir al jugador muerto y que al revivir este se limpia el timer. En cuanto a la función de recargar esta llama a un evento que se

ejecuta en el servidor que comprueba si se ha gastado alguna bala para poder recargar así como si aún quedan cargadores para poder recargar y ahí suma las 10 balas y resta un cargador. Pasando a la función que suma un cargador al acertar dos balas seguidas, la cual se ejecuta en el servidor, es aquí donde el ID que se le pasa a los proyectiles, pues compara el último ID que colisionó con un jugador con el que acaba de recibir, y si son consecutivos entonces suma un cargador extra al jugador siempre que no se sobrepase el máximo de cargadores que son 4 sumado a las 10 balas del cargador ya insertado, cumpliendo así la parte lógica del requisito número 5 (el resto será explicado cuando se llegue a la UI), junto a esto se le añade energía al jugador en caso de que el tipo de proyectil le de dicha energía al jugador (en este proyecto solo la otorga el proyectil primario), dado que si fuese el disparo el que otorga la energía entonces los jugadores se esconderían hasta tener el proyectil secundario cargado, haciendo más aburrido el gameplay, junto a esto también se activa un timer para una comprobación al final con el fin de evitar errores, con esto se tiene la energía que era lo que faltaba del tercer requisito. Finalmente está la acción de tomarse una poción que aumenta en 70 puntos la vida del jugador, la cual se activa al click central del ratón y sólo cuando se tiene el 30% de la vida o menos, esta acción llama a un evento en el servidor que comprueba si el timer entre pociones (de 60 segundos) ha acabado y en caso afirmativo aumenta la salud y vuelve a poner el timer, completando así el octavo requisito y acabando ya con el blueprint del jugador.

Pasando al player widget blueprint la salud se representa mediante un texto que coge la variable de vida actual y la vida máxima y las muestra, así como una barra de progreso de la vida que muestra la salud normalizada. Luego está el tipo de daño que es una imagen que accede al index del tipo de daño que va a hacer el jugador cuando dispare y lo aplica en un array de imágenes para que vea el tipo de daño actual. También un texto que muestra las balas actuales y el máximo de balas, así como una imagen de unas balas para indicar qué representa ese número, lo cual ocurre igual para la energía y los cargadores. Al llegar a las 0 balas se muestra un texto que indica los botones a los que se puede recargar cumpliendo así los requisitos que faltaban del apartado número 5. Junto a esto cuando la energía llega a 70 o más entonces se muestra que el disparo secundario está disponible. Sumado a esto se muestra un sprite de una poción junto a un timer que muestra cuántos segundos quedan hasta poder volver a tomar la poción, acabando así con el séptimo punto los requisitos. También hay un filtro para cuando se recibe daño de algún tipo especial que muestra una imagen por medio segundo, así como un filtro de daño general, el cual se va volviendo más rojo cuanto menor es la salud del jugador. Finalmente cuando el jugador muere la pantalla se pinta de rojo y se muestra un texto que le indica al jugador que ha muerto y el tiempo que queda hasta que respawnee, cumpliendo así el sexto requisito de la entrega.

Pasando al proyectil primario cuando colisiona con un jugador se aplica un point damage y se le pasa el actor dañado, el daño base que se le ha realizado, la dirección desde donde viene el golpe, la información del propio golpe, el instigador, el causante del daño como la propia bola y la clase del tipo de daño que se le ha de aplicar, tras esto se coge al jugador que disparó el proyectil y se llama al evento de enemy hitted (que es el que añade energía y comprueba los disparos consecutivos) y se le pasa por parámetros el ID de la propia bola y si debe o no añadir energía al jugador que la disparó (en el caso del proyectil primario si que añade), finalmente se destruye el proyectil.

En cuánto al proyectil secundario cuando colisiona aplica un radial damage y se le pasa el daño base, el punto dónde se ha generado el golpe, el radio del daño, y el causante del daño como la propia bola y la clase del tipo de daño que se le ha de aplicar (en el proyectil secundario es el daño especial), tras esto se coge al jugador que disparó el proyectil y se llama al evento de enemy hitted y se le pasa por parámetros el ID de la propia bola y si debe o no añadir energía al jugador que la disparó (en el caso del proyectil secundario no añade), finalmente se destruye el proyectil. Este proyectil posee una escala más grande (0'3 en lugar de 0'1) y la esfera hace block all en su colisión para que las cajas se rompan con el fracture, cumpliendo así el cuarto punto de la entrega.

Pasando a la poción extra que hice que se puede recolectar por el mundo, esta sube hasta 30 puntos de vida al jugador que colisiona con ella y luego destruye el actor de la poción.

Finalmente también están los blueprints de los daños, teniendo el normal, el cual ya lo trae Unreal, el daño de fuego, el de hielo y el especial.

En cuanto a los inputs se han añadido inputas para cambiar el tipo de daño que se va a infligir, así como otro para disparar el proyectil secundario, otro para recargar y otro para usar la poción de curación. Junto a esto se agregaron controles para zurdos.

Se añadieron varios sprites para usar en la UI, tanto para los indicadores como para los filtros.

Se añadieron también varios materiales, los dos de los proyectiles, los cuales son de distintos colores, y un master material que hice para el mapa de la template de Unreal junto a varios material instances, que pese a no usar ya ese mapa sigue teniendo dichos materiales asignados.

Se creó también un objeto de tipo geometry collection para los objetos destructibles y una float curve para la pitch del first person character blueprint.

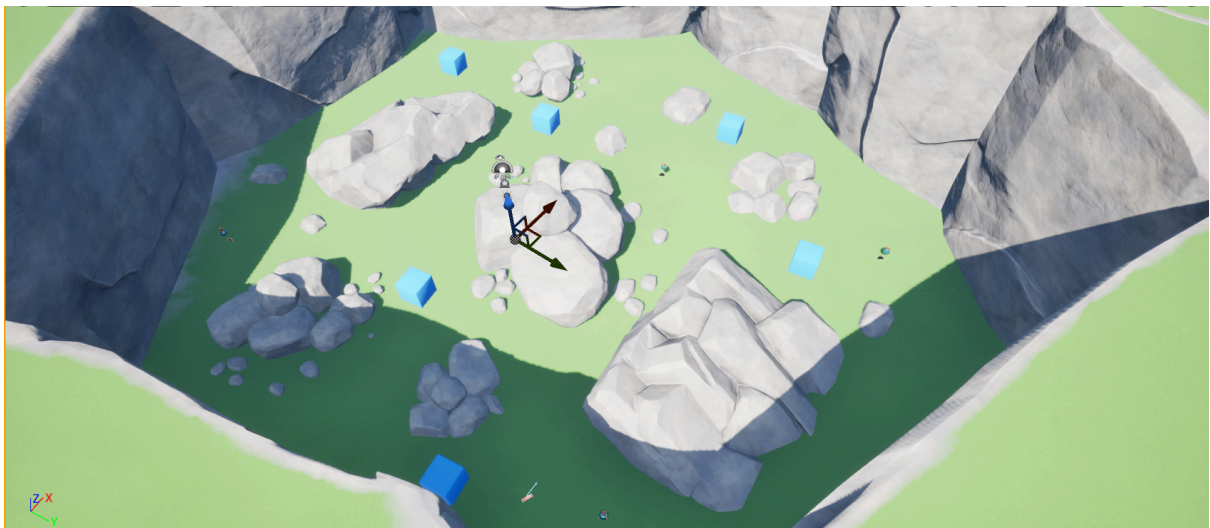
Como mencioné anteriormente ahora se usa otro mapa en vez del de la template de Unreal, dicho mapa es el de una montaña y lo monté usando distintos assets.

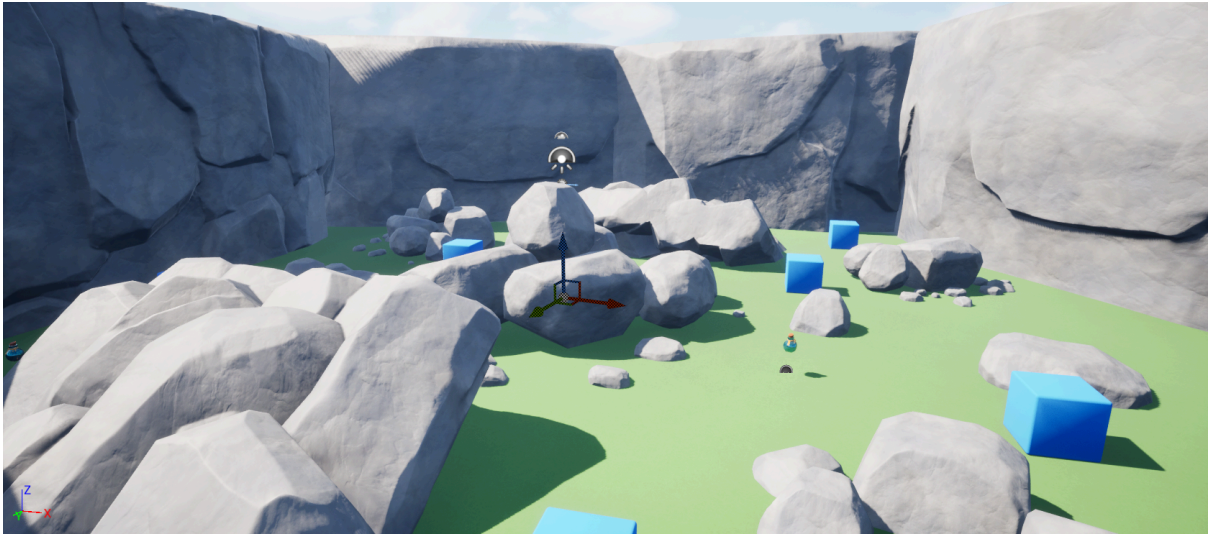
Finalmente están los assets añadidos, desde el modelo de la poción que se pueden recolectar por el mapa así como un pack de assets que compramos el año pasado para el juego de taller que fueron reutilizados para esta entrega y son con los que monté la escena de la montaña.

Diario de desarrollo

Durante el desarrollo de esta práctica ha habido varias complicaciones dado que al implementar la lógica multijugador de Unreal se me complicó un poco de entender y había cosas que no me funcionaban y no sabía muy bien por qué ocurrían, me fue bastante difícil durante la mayoría del tiempo, pero cuando por fin conseguí entender la idea detrás de ello y el cómo funciona la replicación ya logre sacar el resto de la práctica. Otro aspecto que me ha dado bastantes problemas es a la hora de trabajar con el fracture, que me daba muchos fallos al configurarlo, pero finalmente haciendo el proyectil secundario más grande se arreglaron.

Anexos





Bibliografía

- Vídeos explicativos sobre el multijugador en Unreal de Luis Rubio del campus
- Vídeo explicación sobre el fracture de Unreal -> <https://youtu.be/UNUHdJEuzp8?si=fvn4TMAbAbA7Am9C>